Il codice spiegato facile

Aggiornato: 6 novembre 2026, dopo la pulizia (le cartelle week1-week4 sono diventate una sola, `src/analysis/`).

Questo file spiega **a cosa serve ogni file e cosa fa, passo per passo**, con parole semplici. Non serve leggerlo tutto: cerca il file che ti interessa.

---

1. La mappa in 30 secondi

```
src/
  analysis/     le analisi della tesi (12 file)
  report/       le schede del report per l'allenatore (13 file, compreso common.py)
  adapters/     convertono i dati di altri fornitori (Wyscout, CSV) nel nostro formato (3 file)
  tracking/     prove esplorative con i dati di tracking DFL (3 file)
  oppreport.py  un solo comando che fa tutto il report
archive/        codice già usato una volta, che non serve per rifare i numeri (6 file, vedi sezione 6)
tests/          controlli: i numeri escono sempre uguali?
app/            l'app web
results/        i risultati (JSON), mai modificati a mano
docs/           protocolli e spiegazioni
```

Il percorso dei dati è sempre lo stesso:

1. **Scarico** le partite (`download_data.py`).
2. **Taglio** ogni partita in sequenze di passaggi (`sequences_clustering.py`).
3. **Raggruppo** le sequenze simili in famiglie (`families.py`).
4. **Controllo** sulle partite tenute da parte se le famiglie reggono (`validate_replication.py` e gli altri).
5. **Do un voto** A-D a ogni frase del report (`evidence_grade.py`).

---

2. Cinque idee che tornano in tutti i file

Se capisci queste cinque cose, capisci quasi tutto il codice.

a) Partite di esplorazione e partite riservate. Le partite sono ordinate per data. Il primo 70% serve per guardare e scegliere (esplorazione). L'ultimo 30% resta chiuso in un cassetto e si apre una volta sola, per il test vero (partite riservate). Nel codice: `k0 = int(round(n_matches * 0.7))` e poi `df.gi < k0` (esplorazione) oppure `df.gi >= k0` (riservate). `gi` è il numero della partita in ordine di data.

b) Le due fasi. Una sequenza che parte prima di x = 45 (nella nostra metà campo, su un campo lungo 120) è **costruzione**. Se parte da 45 in avanti è **fase avanzata**. Nel codice: `d.x0 < 45`.

c) Le 11 caratteristiche (`COLS`). Ogni sequenza diventa 11 numeri: dove inizia (`x0`, `w0`), dove finisce (`x1`, `w1`), quanto avanza (`net_x`), quanta strada fa la palla (`path_len`), quanto va dritta (`directness`), quanti passaggi (`n_pass`), quota di lanci lunghi (`long_share`), quanto si allarga (`width_std`) e quanto dura (`dur`). `w0` e `w1` sono la distanza dal centro del campo (`abs(y - 40)`).

d) K-Means. È l'algoritmo che fa i gruppi: mette insieme le sequenze con numeri simili. Prima i numeri si mettono sulla stessa scala (`StandardScaler`), sennò i metri peserebbero più dei secondi. `KMeans(5, n_init=20, random_state=0)` vuol dire: 5 gruppi, prova 20 partenze diverse e tieni la migliore, con il seme fisso così il risultato è sempre uguale.

e) Bootstrap per partite. Serve a capire quanto un numero è sicuro. Si pescano a caso le partite (con ripetizione), si ricalcola il numero, e si fa 1000 volte. Il 95% dei risultati sta nell'**intervallo di confidenza** (IC95). Se l'intervallo è tutto sopra zero, l'effetto è "chiaro". Nel codice lo vedi sempre così:

```
for _ in range(1000):
    pick = rng.choice(games, len(games))       # pesco le partite
    means.append(np.mean(...))                 # ricalcolo
ci = np.percentile(means, [2.5, 97.5])         # intervallo al 95%
```

`rng = np.random.default_rng(2026)` è il generatore di numeri casuali con un **seme fisso**: ogni volta escono gli stessi numeri, quindi i risultati si possono rifare identici.

---

3. `src/analysis/` : le analisi della tesi

`download_data.py` : scaricare i dati

A cosa serve: scarica da GitHub i dati StatsBomb di un campionato. **Come si usa:** `python download_data.py 9 281 "Bayer Leverkusen"` (9 = Bundesliga, 281 = stagione 2023/24). **Cosa fa:** 1. Crea le cartelle `matches/` (elenco partite), `ev/` (eventi) e `t360/` (posizioni dei giocatori, quando ci sono). 2. Scarica l'elenco delle partite e tiene solo quelle della squadra. 3. Scarica gli eventi di ogni partita, 8 file alla volta. 4. Se un file c'è già, non lo riscarica: se si interrompe, basta rilanciarlo.

`sequences_clustering.py` : le sequenze

A cosa serve: trasforma ogni partita in una tabella di sequenze. **Quasi tutti gli altri file lo usano** (`import sequences_clustering as S` , poi `S.extract()`). **Cosa fa `extract()` :** 1. Prende le partite della squadra e le ordina per data. 2. Per ogni partita raggruppa i passaggi, le conduzioni, i tiri e i dribbling della squadra per **possesso** (il numero di possesso lo dà StatsBomb). 3. Per ogni possesso chiama `sequence_row()` , che lo scarta se non è gioco aperto (niente angoli, punizioni, calci d'inizio) o se ha meno di 3 passaggi. 4. Se lo tiene, calcola i numeri della sequenza: inizio, fine, lunghezza, durata, se finisce con un tiro, ecc. 5. Restituisce una tabella (una riga = una sequenza) e il numero di partite.

Altre parti: `zone()` dice in quale delle 12 zone del campo cade un punto (4 fasce in lunghezza A-D × 3 in larghezza: s = sinistra, C = centro, d = destra). `jaccard_stability()` misura se i gruppi si ritrovano rifacendo K-Means su dati ripescati. La scelta del numero di gruppi (k), fatta una volta sola, ora è in `archive/choose_k.py` .

`families.py` : le famiglie (solo partite di esplorazione)

Come si usa: `python families.py final|robustness|null <partite.json> "<Squadra>"` - **`final`** : il metodo finale. In ogni fase fa K-Means (5 famiglie in costruzione, 3 in avanzata) e dà i nomi C1-C5 e A1-A3, ordinati da quella che parte più indietro. Per ogni famiglia scrive il profilo (quota, passaggi, lanci lunghi, tiri, percorsi tipici fatti di 3 zone) e disegna una figura con un campo per famiglia. - **`robustness`** : cambiano le famiglie se cambio il taglio (35-55 invece di 45), il numero k o l'algoritmo (mistura gaussiana, Ward)? E si riesce a prevedere la famiglia della prossima sequenza? - **`null`** (test E1): fa lo

stesso controllo di stabilità su dati finti **senza gruppi** (colonne mescolate, oppure una nuvola gaussiana). Se le famiglie vere sono molto più stabili dei dati finti, non sono un'illusione di K-Means.

`validate_replication.py`: il test vero (H1, H2, H3)

Come si usa: `python validate_replication.py <partite.json> "<Squadra>" . --confirm` (aggiungi `--no-h3` per Leverkusen e Leicester, validate prima che H3 esistesse). **Cosa fa:** 1. Divide le partite: 70% per allenare, 30% riservate. 2. Trova le famiglie **solo** sul 70% e poi le "congela". 3. Dà a ogni sequenza delle partite riservate la famiglia più vicina. 4. **H2:** le quote delle famiglie cambiano poco (al massimo 10 punti) e i gruppi restano separati (silhouette almeno 0.10)? 5. **H1:** sapendo come inizia la sequenza, il minuto, il tempo e la famiglia precedente, indovino la famiglia meglio che usando solo le quote? Si misura col **log-loss** (più basso = previsione migliore). Guadagno = log-loss di base meno log-loss del modello. H1 passa se l'IC95 del guadagno è tutto sopra zero. 6. **H3:** sapere la famiglia aiuta a prevedere se la sequenza finisce con un tiro? 7. Salva tutto in `holdout_<Squadra>.json`.

Senza `--confirm` non apre le partite riservate: è una protezione, perché il test si fa una volta sola.

`compare_simple_rules.py`: le famiglie battono una regola banale?

Cosa fa: confronta le famiglie con una regola semplicissima, cioè dividere le sequenze in fasce in base a **dove finiscono** (x1). Per entrambe controlla quanto bene prevedono il tiro sulle partite riservate. Risultato: in nessuna squadra le famiglie fanno meglio (risultato negativo, dichiarato nella tesi).

`sample_size.py`: quante partite servono?

Cosa fa: pesca a caso 3, 5, 8, 10 o 15 partite (500 volte) e guarda quanto i numeri di quelle poche partite si allontanano dai numeri di tutta la stagione. Così si capisce da quante partite in poi un report è affidabile.

`ten_teams.py`: tutto sulle 10 squadre

Come si usa: - `run simplerule|samplesize|clustertest`: lancia uno script su tutte le 10 squadre (salta quelle già fatte). - `summary replication|simplerule|samplesize`: stampa la tabella finale e la regola del protocollo (almeno 7 squadre su 10).

L'estrazione a sorte delle 6 squadre e il primo lancio della replica (fatti una volta sola) sono in `archive/select_and_replicate.py`.

`exploratory_predictability.py` (esplorativo): da dove viene la prevedibilità?

Usa tutte le partite, con la validazione incrociata (6 gruppi di partite: alleno su 5, provo sul sesto, e giro). - `cv`: il guadagno del modello H1 su tutte le partite. - `ablation`: tolgo un ingrediente alla volta. Risultato: quasi tutto il guadagno viene da **come inizia la sequenza** (play pattern). - `context`: a parità di inizio, il minuto, il punteggio o casa/trasferta aiutano? Risultato: no.

`exploratory_clusters.py` (esplorativo): le famiglie sono gruppi veri?

- `copula`: confronta i dati veri con 100 dati finti che hanno le stesse distribuzioni e correlazioni ma **nessun gruppo**. Risultato: le famiglie sono solo un po' più nette dei dati finti, cioè "fette di un continuo".
- `methods`: K-Means contro altri due algoritmi, e con altri k o tagli.

`corners.py` : gli angoli (L3)

- `leverkusen` : dove manda gli angoli il Leverkusen? Usando le posizioni 360 dei difensori (più vicini o più lontani dal battitore), una difesa potrebbe sfruttarlo? Il test di permutazione mescola gli xG a caso 1000 volte per vedere se l'effetto è più grande del caso. Risultato: p = 0.17, niente di chiaro.
- `pooled` : lo stesso su Mondiale 2022 + Europei 2020 e 2024 insieme (846 angoli). Risultato: p = 0.41.

`evidence_grade.py` : il voto A-D (il contributo principale)

Idea: ogni frase del report ("il 40% dei rinvii va corto", "PPDA 12.6", ...) riceve un voto in base a due domande: 1. **Previsione:** il numero calcolato sulle partite passate prevede la **prossima** partita meglio della media del campionato? 2. **Discriminazione:** il numero distingue davvero le squadre, o le differenze sono solo rumore?

A = forte su entrambe, D = nessuna delle due. **Come si usa (due passi; la prova "smoke" fatta prima del protocollo è in `archive/smoke_grade.py`):** - `extract` : per ogni squadra e ogni partita calcola numeratore e denominatore di ogni frase, **usando gli stessi script del report** (così il voto vale proprio per quello che il report scrive). Salva un file `.pkl` per squadra. - `analyse` : calcola i voti. Per ogni frase fa le previsioni partita per partita (dalla 6ª in poi), misura la previsione (skill), la discriminazione (D), quante volte l'intervallo all'80% contiene la partita vera, e quante partite servono.

Le funzioni principali: `match_values` (i numeri di una partita), `all_forecasts` (tutte le previsioni), `ratio_ci` (un numero con il suo intervallo, ripescando le squadre), `discrimination` , `grade` (la tabellina che trasforma le due risposte in una lettera).

`evidence_grade_wyscout.py` : RQ1, i voti su Wyscout

Stesso codice di `evidence_grade.py` (lo importa), ma con i dati Wyscout 2017/18 (5 campionati, 98 squadre). Toglie le 5 frasi che Wyscout non può calcolare (pressioni, xG) e alla fine confronta lettera per lettera i voti StatsBomb con quelli Wyscout.

`forecast_opponent.py` : RQ2, l'avversario aiuta la previsione?

Prende la previsione del voto (la media della stagione finora) e la corregge in base all'**avversario** (cosa hanno fatto le altre squadre contro di lui) e al **campo** (casa o trasferta). Poi guarda se la previsione corretta sbaglia meno. `dev` sceglie quanto correggere sulle 9 squadre di sviluppo; `confirm` fa il test una volta sola su squadre nuove.

Due fornitori, stesse partite (ora in `archive/`)

`evidence_provider.py` e `exploratory_provider_samematch.py` prendono le 36 partite del Barcellona 2017/18 che esistono sia in StatsBomb sia in Wyscout e confrontano i numeri. Risultato: le famiglie e i passaggi per sequenza sono simili; altri numeri (lanci lunghi, rinvii corti) no. Per questo i valori non si confrontano tra fornitori diversi, i voti sì.

---

4. `src/report/` : le schede del report

Ogni file fa **una scheda** per una squadra: una pagina HTML da stampare, un JSON con tutti i numeri (lo legge l'app) e spesso una **lista di clip** (CSV) con il minuto di ogni azione. Si lanciano tutti insieme con `oppreport.py` . Funzionano tutti allo stesso modo:

1. leggono l'elenco delle partite e gli eventi della squadra;
2. per ogni partita contano quello che serve (per esempio quanti rinvii vanno corti);
3. sommano sulla stagione e calcolano un intervallo con il bootstrap per partite;
4. guardano anche le **ultime 5 partite**, per vedere se la squadra sta cambiando;
5. scrivono la pagina HTML e il JSON.

Le regole (cosa conta come "corto", come "palla persa", ecc.) sono scritte all'inizio di ogni file e sono state fissate **prima** di guardare i risultati.

`make_report.py` : il report principale

Usa il metodo della tesi con le stesse impostazioni (taglio 45, k = 5 e 3), ma su **tutta** la stagione: qui si descrive, non si testa. 1. Prende le sequenze (`S.extract()`) e calcola lo stile: passaggi per sequenza, lanci lunghi, quota dalla propria metà, tiri. Ogni numero ha il suo intervallo e il confronto con le altre squadre della cartella. 2. Trova le famiglie in ogni fase e, per ognuna, scrive quota, profilo, tasso di tiro, quanto varia da partita a partita e quanto è stabile. 3. Fa il controllo di struttura (le famiglie sono più nette di dati senza gruppi?) e la tabella "come inizia la sequenza → in che famiglia finisce". 4. Scrive "quanto fidarsi di questo report" in base al numero di partite. 5. Scrive anche `clips_<Squadra>.csv` (ogni sequenza con famiglia e minuto) e il **foglio di revisione** per l'analista (`review_<Squadra>.csv/.html` : 3 esempi per famiglia e cinque domande fisse).

`restarts.py` : le ripartenze

Rinvii dal fondo, rimesse del portiere con le mani e rimesse laterali (nella propria metà e in quella avversaria). Per ognuna guarda il **primo passaggio**: corto (meno di 30 m) o lungo, verso sinistra, centro o destra, e chi lo riceve. Poi guarda come finisce il possesso: arriva nella metà avversaria? Nell'ultimo terzo? Finisce nella propria metà? Con un tiro? Disegna anche un campo con le frecce dei primi passaggi.

`setpieces.py` : angoli e punizioni

Per gli **angoli**: lato, zona di arrivo (corto/fuori area, primo palo, centro, secondo palo: le stesse zone di `corners.py`), chi batte, se nasce un tiro e quanti xG. Per le **punizioni** nella metà avversaria: tiro diretto, cross in area o gioco corto.

`transitions.py` : palle perse e recuperate

Non usa i possessi di StatsBomb, ma solo l'**ordine delle azioni**, così funziona con qualunque fornitore. - **Palla persa:** una nostra azione seguita da almeno 2 azioni di fila dell'avversario (o da un suo tiro). - **Contropressing:** la riprendiamo entro 5 secondi. - **Tiro subito:** l'avversario tira entro 10 secondi. - **Recupero:** il contrario, cioè dove recuperiamo palla e se tiriamo entro 10 secondi.

Conta tutto per terzo di campo e disegna le mappe. Nella scheda c'è scritto onestamente che il contropressing è al 17-19% per tutte le squadre, quindi non distingue gli stili.

`defending.py` : come difende

PPDA (quanti passaggi lasciamo fare all'avversario nella sua zona per ogni nostra azione difensiva: basso = pressing alto), pressioni per zona e per terzo, altezza media delle azioni difensive, tiri e xG subiti. Ha bisogno degli eventi difensivi di StatsBomb (Pressure, Duel...): con il CSV generico si ferma e lo dice.

`matchups.py` : contro chi va meglio o peggio

Divide le partite passate in due gruppi: avversario che pressa tanto o poco (PPDA sopra o sotto la mediana), difesa a 3/5 o a 4, casa o trasferta. Confronta i numeri dei due gruppi con un intervallo e dice "chiaro", "incerto" o "gruppo troppo piccolo" (meno di 6 partite). **Solo descrittivo:** non dice quale modulo usare.

`keyplayers.py` : i giocatori chiave

I primi 3 giocatori per ruolo: chi riceve in costruzione, chi entra nell'ultimo terzo, chi fa più xG, chi batte angoli e punizioni, chi perde più palloni nella propria metà. Fa anche la linea del tempo dei gol (fatti e subiti ogni 15 minuti); ho controllato che i gol corrispondano ai risultati veri delle partite.

`xt_model.py` + `xthreat.py` : la pericolosità (xT)

- `xt_model.py` si lancia **una volta sola**. Divide il campo in 16 × 12 zone e calcola per ogni zona la probabilità che, avendo la palla lì, il possesso finisca in gol (modello di Karun Singh). Salva la griglia in `xt_grid.json`.
- `xthreat.py` usa quella griglia. Per ogni passaggio o conduzione riuscita, xT aggiunto = valore della zona di arrivo − valore della zona di partenza. Somma per partita e mostra da dove parte la minaccia, chi la crea e quanta ne concediamo.

`summary.py` : la pagina di preparazione

Una sola pagina da stampare, con i fatti principali di tutte le schede. Accanto a ogni numero c'è la posizione in classifica rispetto alle altre squadre della cartella. Non calcola niente di nuovo: legge i JSON delle altre schede.

`clips_to_xml.py` : le clip per il video

Trasforma una lista di clip in un file XML per i programmi video (Sportscode, Nacsport), uno per partita. Deve sapere a che secondo del video inizia ogni tempo (file degli offset).

`common.py` : le funzioni in comune

Le piccole funzioni che prima erano copiate in più schede, ora scritte una volta sola: `team_matches` (le partite della squadra in ordine di data), `opponent` (chi è l'avversario), `third` (terzo di campo), `boot` e `boot_stat` (intervallo al 95% ripescando le partite), `pc` e `ci` (scrivono 0.23 come "23%" e [0.21, 0.27] come "21-27%"), `fig_png` (trasforma un grafico in un'immagine dentro la pagina).

`summarize_review.py`

Legge il foglio di revisione compilato dall'analista (sì / in parte / no per ogni sequenza) e conta, per famiglia, quante sequenze l'analista trova coerenti.

---

5. Gli altri file

`src/oppreport.py` : un comando per tutto

1. Guarda cosa gli hai dato: un CSV (`--csv`) o i dati StatsBomb (`--statsbomb`).
2. Se è un CSV, lo converte prima con `generic_csv.py` .
3. Se mancano degli eventi, si ferma e ti scrive il comando per scaricarli.
4. Lancia le schede una dopo l'altra. Le prime quattro devono riuscire; le altre quattro hanno bisogno di eventi StatsBomb e, se falliscono, vengono saltate con un messaggio.
5. Fa la pagina di preparazione e copia tutto nella cartella `--out` . Con `--xml` prepara anche i file per il video.

`src/adapters/` : altri fornitori di dati

- `generic_csv.py` : legge un CSV semplice (una riga per evento: partita, minuto, squadra, tipo, x, y, ...) e lo riscrive come se fosse StatsBomb, così tutto il resto del codice funziona uguale. Se il CSV non ha i numeri di possesso, li ricostruisce: un nuovo possesso inizia quando cambia il tempo, a ogni ripartenza, o quando l'altra squadra fa 2 azioni di fila (o tira). Con `--validate-only` controlla solo il file e dice cosa manca.
- `wyscout_to_generic_csv.py` : trasforma i dati pubblici Wyscout in quel CSV. Le regole di traduzione (cosa è un passaggio, una rimessa, un autogol, ecc.) sono scritte in cima al file.
- `statsbomb_to_generic_csv.py` : il contrario (StatsBomb → CSV). Serve solo per il test di andata e ritorno.

`src/tracking/` : dati di tracking (esplorativo, altre partite)

Dati DFL: 7 partite tedesche con la posizione di tutti i giocatori 25 volte al secondo. - `dfl_load.py` : legge i file XML della DFL e li salva in un formato più veloce (`.npz`). - `pitch_control.py` : per ogni punto del campo calcola quale squadra ci arriverebbe prima (modello di Spearman, parametri presi dalla letteratura). Controllo: prevede bene quali passaggi riescono (AUC 0.89). - `ghosting.py` : prova a prevedere dove sarà ogni difensore tra 3 secondi, con una rete neurale (LSTM) contro modelli semplici. Risultato onesto: la regressione lineare (ridge) va meglio della rete neurale.

`tests/` : i controlli

- `reproduce_results.py` : rilancia le analisi della tesi in una cartella nuova e confronta i 51 risultati con `results/` . Devono essere **identici**. Da fare prima di riportare un numero al professore.
- `reproduce_sheets.py` : lo stesso per le schede (difesa, matchup, giocatori, transizioni, xT).
- `test_adapter_roundtrip.py` : StatsBomb → CSV → di nuovo StatsBomb. Se le sequenze escono uguali, l'adapter funziona.
- `test_app_api.py` : controlla che l'app carichi ogni squadra e che i numeri mostrati siano quelli dei file.
- `test_all_teams.py` : chiama ogni pagina dell'app per ogni squadra; nessuna deve dare errore.
- `check_clips.py` e `check_replays.py` : controllano che ogni clip mostri davvero l'azione giusta (stesso minuto, stessi passaggi, tiro quando la lista dice tiro) e che i replay non disegnino cose strane (giocatori fuori dal campo, passaggi di lunghezza zero).

`app/` : l'app web

- `app/run.py` : avvia l'app su http://127.0.0.1:8765.
- `app/backend/server.py` : il "cameriere" dell'app. Quando la pagina chiede qualcosa (i numeri di una squadra, un replay, una lista di clip), legge i file prodotti dalla pipeline e li manda. Non calcola stime nuove.
- `app/backend/gameplan.py` : il piano partita. Regole fisse: se la squadra è tra le prime (o le ultime) per un numero, scatta un suggerimento su **dove guardare** nel video, con i numeri che l'hanno fatto scattare. Non sono consigli tattici testati.

- `app/backend/ai_brief.py` : facoltativo. Un modello linguistico **locale** (Ollama, sul tuo PC, gratis) riscrive i suggerimenti in un breve testo per l'allenatore. Non aggiunge niente, e il codice controlla che i numeri siano quelli giusti.
- `app/backend/tracking.py` : la sezione "Tracking lab" dell'app (replay delle partite DFL con il pitch control).
- `app/frontend/` : la parte grafica (React). Ogni file in `views/` è una pagina dell'app.

---

6. `archive/` : quello che non serve più

Codice già usato una volta. Non serve per rifare i numeri della tesi, ma resta qui così ogni risultato ha ancora il suo codice. È lo stesso codice di prima, solo spostato.

file	cosa ha fatto
`choose_k.py`	ha scelto il numero di famiglie (k = 5 e 3) sulle partite di esplorazione
`select_and_replicate.py`	ha estratto a sorte le 6 squadre nuove e ha fatto il primo lancio della replica
`smoke_grade.py`	prove del codice del voto prima del test vero (solo conteggi)
`evidence_provider.py` , `exploratory_provider_samematch.py`	il confronto StatsBomb contro Wyscout sulle stesse 36 partite
`clips_to_mp4.py`	taglia clip MP4 da un video (non lo usano né il report né l'app)

---

7. Parole difficili, in una riga

parola	vuol dire
silhouette	quanto i gruppi sono separati (da -1 a 1; vicino a 0 = gruppi che si toccano)
Jaccard	quanto un gruppo si ritrova uguale rifacendo l'analisi (1 = identico)
ARI	quanto due divisioni in gruppi sono d'accordo (1 = identiche)
log-loss	quanto una previsione sbaglia (più basso = meglio)
regressione logistica	il modello che dà la probabilità di ogni famiglia (o del tiro)
bootstrap	ripescare i dati a caso tante volte per vedere quanto un numero può cambiare
IC95	l'intervallo in cui sta il numero vero con 95% di fiducia
seme (seed)	il numero che fissa il caso: stessi numeri casuali ogni volta
permutazione	mescolare i dati per vedere cosa succede "per caso"
xG	probabilità che un tiro diventi gol
PPDA	passaggi concessi all'avversario per ogni azione difensiva (basso = pressing alto)